# REPORT

# 程序运行

`xmake build` 编译
`xmake run demo` 运行

---

# 题意理解

从数学规划看，将$u$对$t$离散化得到$t$的格点函数$\hat{u}$。对于一个固定的$\hat{u}$，可以用数值方法（求解IVP的方法如LMM，One-Step Method）求出一个$J$。通过优化方法选择合适的$\hat{u}$则可以得到$J$的最大值。

所以主要分为两部分。第一部分是求解IVP，第二部分是优化。

---

# 数值方法

一开始选择了显性的经典RK法，虽然有四阶精度，但是在将$t_f$离散化为1000段时，计算结果一致在小于1处无法提升。而在离散化为10000段时较精确。

自适应步长的RK方法，出乎意料的算的慢，说明可能确实函数总体对精度要求较高，我用四阶方法和五阶方法的误差作为

变换步长的依据，所以可能一直在缩小步长，从而也导致不增加patience值很容易提前停止（或者把最小步长调大一些）。

五阶的RK方法，容易在计算时略微超过最大的J的理论值后，飙升至正无穷。当然这也有可能是我的优化算法不够好或超参数没调对。

## 程序架构

使用单例模式和工厂模式管理求解器，在solver.h文件中。

在story.h文件中完成求解器的注册和问题相关函数的定义。

---

# 优化算法

## 粒子群算法

步骤1：初始化，创建n个粒子，并且给予每个例子p坐标，v速度。每个粒子的pBest就是初始化位置，gBest就是所有粒子中J最大粒子所在的位置。

步骤2：循环进化，所有粒子按照以下两个公式进化，其中$w$为惯性因子，动态变化可以使算法更高效，$c_1, c_2$为认知因子，$\theta$为惯性因子

$$
\begin{aligned}
v_i &= wv_i && + c_1 \times rand() \times (pBest_i - x_i) \\
& && + c_2 \times rand() \times (gBest_i - x_i) \\
x_i &= x_i + \theta v_i
\end{aligned}
$$

进化完后依此更新pBest和gBest。（所以要保存位置和J）
若长时间全局适应度最大值不变化，随机重置一部分粒子。

步骤3：达到最大进化步数或若干步变化很小时停止循环。

代码在pos.h中，相关参数意义有注释。

---

# 数值结果

相关参数在story.h中修改

四阶RK方法+分成10000段

```
step: 54, gBestval: 0.476502
step: 55, gBestval: 0.476502
step: 56, gBestval: 0.476502
step: 57, gBestval: 0.476502
step: 58, gBestval: 0.476502
Early stopping at step 59 due to stagnation.
result: 0.476502
```

五阶RK方法+分成10000段

```
step: 66, gBestval: 0.449206
step: 67, gBestval: 0.454832
step: 68, gBestval: 0.810102
step: 69, gBestval: 1.10455
step: 70, gBestval: 477.735
```

超过理论值后迅速飙升至无穷

自适应步长RK

```
step: 133, gBestval: 0.415109
step: 134, gBestval: 0.415109
step: 135, gBestval: 0.415109
step: 136, gBestval: 0.415109
step: 137, gBestval: 0.415109
step: 138, gBestval: 0.415109
step: 139, gBestval: 0.415109
step: 140, gBestval: 0.415109
step: 141, gBestval: 0.415109
step: 142, gBestval: 0.415109
step: 143, gBestval: 0.415109
step: 144, gBestval: 0.415109
step: 145, gBestval: 0.415109
step: 146, gBestval: 0.415109
step: 147, gBestval: 0.415109
step: 148, gBestval: 0.415109
step: 149, gBestval: 0.415109
step: 150, gBestval: 0.415109
step: 151, gBestval: 0.415109
step: 152, gBestval: 0.415109
step: 153, gBestval: 0.415109
```

跑得很慢